ASSIGNMENT-4

- ## PROBLEM STATEMENT:

A Full Adder should perform:
Sum= $A \oplus B \oplus C\ in$
Cout= $AB + ACin + BCin$

Suppose a design error occurs during implementation.
Use AI to:
1.Analyze the truth table.
2.Identify the fault.
Determine the correct logic.
3.Verify the repaired circuit.

- ## OBJECTIVE:

Design a deliberately faulty digital circuit (Full Adder), use Generative AI to diagnose the fault from its truth table, identify the incorrect logic, correct the circuit, and verify the corrected design through simulation.

- ## GEN AI PROMPT:

I designed a Full Adder circuit, but it has a deliberate fault. Here is the observed truth table from my simulation: A=0, B=0, Cin=0 -> Sum=0, Cout=0 A=0, B=0, Cin=1 -> Sum=1, Cout=0 A=0, B=1, Cin=0 -> Sum=1, Cout=0 A=0, B=1, Cin=1 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=0, Cin=0 -> Sum=1, Cout=0 A=1, B=0, Cin=1 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=1, Cin=0 -> Sum=1, Cout=1 (Error: Sum should be 0) A=1, B=1, Cin=1 -> Sum=1, Cout=1 Please diagnose the fault from this truth table. Explain which logic gate is likely malfunctioning or wired incorrectly in the Sum calculation, and tell me how to correct it.

- **GEN AI RESULT:**

  Based on the truth table you provided, the Cout (Carry-out) logic is working perfectly, but the Sum output is faulty.

  Here is the step-by-step analysis of your observed inputs versus the expected mathematical outputs for a Full Adder:

- **A=0, B=1, Cin=1:** Expected Sum = 0 ($0 \oplus 1 \oplus 1 = 0$). Observed Sum = 1.
- **A=1, B=0, Cin=1:** Expected Sum = 0 ($1 \oplus 0 \oplus 1 = 0$). Observed Sum = 1.
- **A=1, B=1, Cin=0:** Expected Sum = 0 ($1 \oplus 1 \oplus 0 = 0$). Observed Sum = 1.
- **A=1, B=1, Cin=1:** Expected Sum = 1 ($1 \oplus 1 \oplus 1 = 1$). Observed Sum = 1.

The Diagnosis :

Your Sum output is returning a 1 whenever *any* of the inputs (A, B, or Cin) is a 1. This means your Sum path is currently executing an **OR** logic operation ($A+B+Cin$) instead of the required **XOR** (Exclusive-OR) logic operation ($A \oplus B \oplus Cin$).

Depending on how you constructed the circuit in your simulation software, the exact hardware fault is one of two things:

1. **If using multi-input gates:** You accidentally placed a 3-input **OR gate** to calculate the final Sum instead of a 3-input **XOR gate**.
2. **If built using two Half-Adders:** A standard Full Adder cascades two XOR gates for the Sum ($Sum=(A \oplus B) \oplus Cin$). To get the exact faulty truth table you observed, you likely replaced **both** of those XOR gates with standard OR gates.

How to correct it:

To fix the circuit, trace the wiring backward from the Sum output pin.

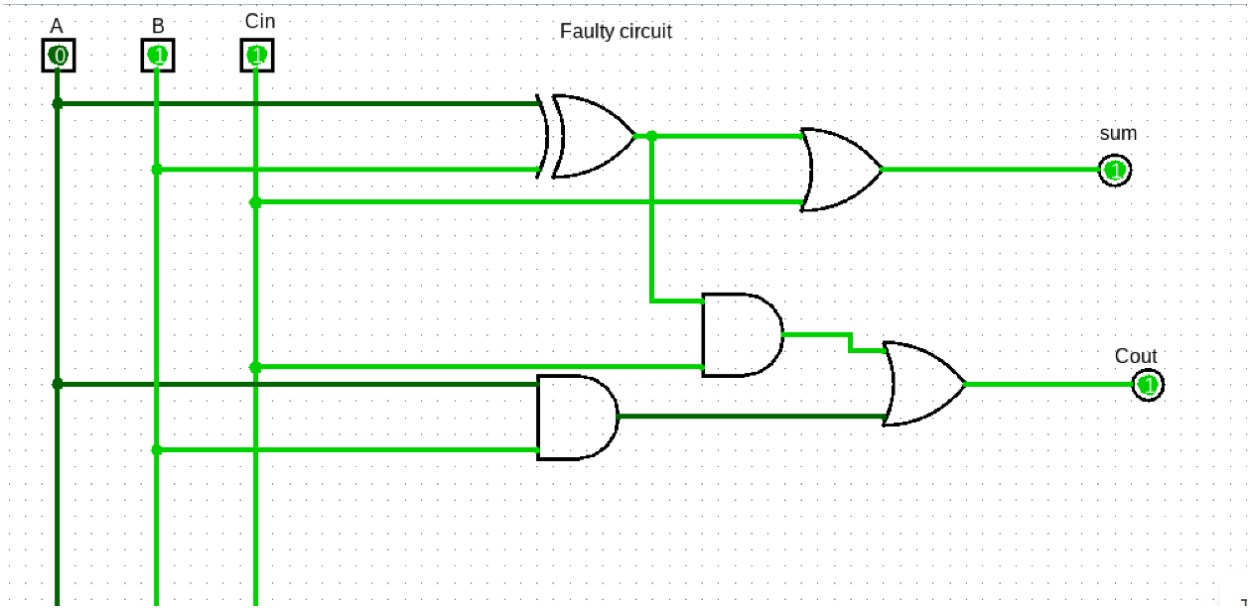
1. Identify the gate(s) computing the final Sum.
2. Delete the OR gate(s) currently in that path.
3. Replace them with **XOR gate(s)** so the boolean equation becomes $Sum=A \oplus B \oplus Cin$. Do not change the gates in your Cout path, as they are already functioning correctly.

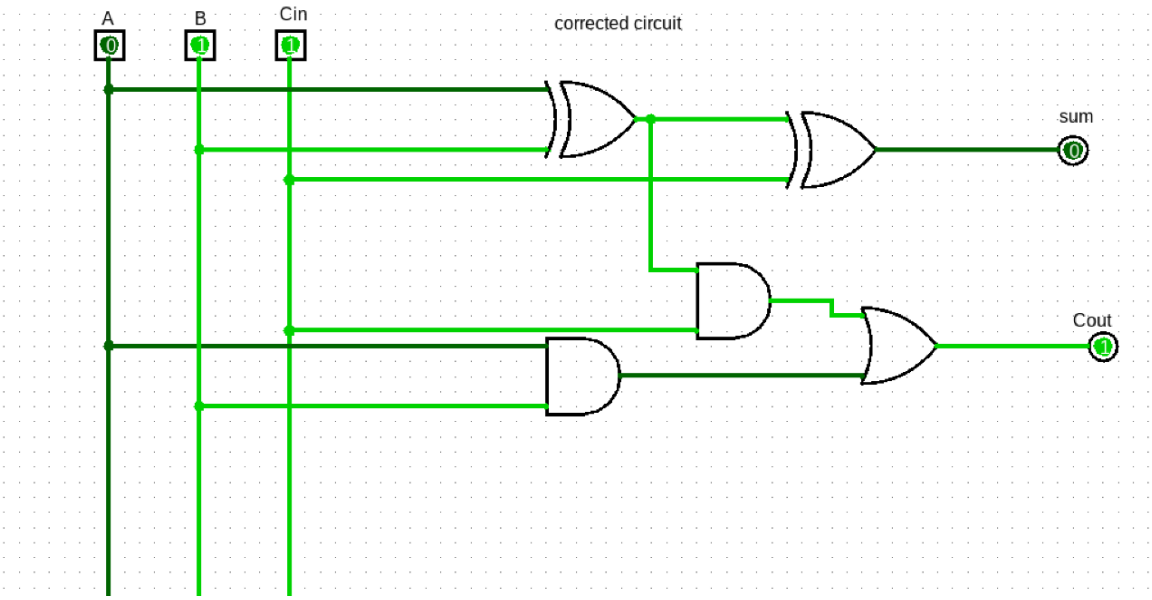
- **TRUTH TABLE FOR BOTH FAULTY AND CORRECTED CIRCUITS:**

A	B	Cin	Correct Sum	Correct Cout	Faulty Sum (Your Circuit)	Faulty Cout (Your Circuit)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	**0**	1	**1**	1
1	0	0	1	0	1	0
1	0	1	**0**	1	**1**	1
1	1	0	**0**	1	**1**	1
1	1	1	1	1	1	1

The bolded **0** values in the "Correct Sum" column highlight exactly where your circuit's OR gate logic failed to execute the required XOR mathematical operation.

- **CIRCUIT IMPLEMENTATION AND LOGISIM SIMULATION**





- ## Discussion and Verification (Reflection)

- **Circuit Verification:**

  Initially, the simulation proved that inserting an OR gate instead of an XOR gate in the second stage of the Sum logic caused the output to saturate at 1 whenever at least one input condition was met. After substituting the correct XOR gate as instructed by the AI diagnosis, the Logisim truth table matched the theoretical Full Adder behavior perfectly across all 8 states.

- **Reflection on GenAI's Output:**

  The GenAI correctly identified the fault by analyzing the specific failure rows. It recognized that the Sum was evaluating to 1 when it should have been 0 specifically in cases where both intermediate inputs to the final gate were 1. By deducing that 1 [Gate] 1 = 1 was occurring instead of 1 XOR 1 = 0, the AI pinpointed the exact location of the hardware bug (an accidental OR gate) without ever seeing the circuit schematic.